Department of Computer Science & Engineering Education

Information Retrieval Techniques

NAME - RAJ

ENROLLMENT NO. - NITRB-25-BD-PG-010

COURSE - M.TECH CSE(BIG DATA ANALYTICS)

GithubLink
https://github.com/rajsanodiya122/Information_Retrieval_Techniques

UNIT 4 · ACTIVITY 8 — PART 2 OF 2

Domain-Specific Mini Search Engine

HITS · Query Interface · Personalisation · Evaluation

Problem Statement

In Activity 7, you built the backend of a mini search engine: a crawler, an inverted index, and PageRank scoring. However, a complete search engine also needs:

- A query interface so users can actually search
- HITS algorithm to discover domain-specific hubs and authority pages
- Query log recording and analysis to understand user behaviour
- Personalization to improve results based on individual user history

In this activity, you will complete your search engine by implementing these components, resulting in a fully functional domain specific mini search engine.

Objective

Extend the Activity 7 backend to develop a complete search engine that:

- Implements HITS algorithm and compares it with PageRank
- Provides a working query interface (CLI or simple Flask/Tkinter UI)
- Combines TF-IDF retrieval scores with link-based ranking (PageRank or HITS)
- Records and analyzes query logs
- Personalizes search results based on user query history
- Evaluates the overall system using precision and recall

1. Introduction 4
2. Task 1 — HITS Algorithm 5
2.1 Algorithm Overview 5
2.2 Execution 5
2.3 Top-5 Hub Pages 6
2.4 Top-5 Authority Pages 7
2.5 PageRank vs HITS Comparison 7
3. Task 2 — Query Interface 8
3.1 Design 8
3.2 Execution Screenshots 8
4. Task 3 — Query Log Analysis 12
4.1 Log Structure 12
4.2 Top-10 Most Frequent Queries 12
4.3 Query Frequency Distribution 13
5. Task 4 — Personalisation 14
5.1 User Profile Model 14
5.2 Comparison: Same Query, Two Profiles 14
6. Task 5 — System Evaluation 16
6.1 Methodology 16
6.2 Results 16
7. End-to-End Pipeline Demo 18
8. Conclusion 19

1. Introduction

This report documents Activity 8, the second and final part of the Domain-Specific Mini Search Engine project. Building directly on the crawler, inverted index, and PageRank implementation from Activity 7, this activity extends the system into a fully functional search engine with four additional capabilities: the HITS algorithm for hub and authority discovery, a command-line query interface with combined TF-IDF and PageRank ranking, query log recording and frequency analysis, result personalisation based on user history, and a formal precision and recall evaluation.

The domain remains Information Retrieval. All code was written in Python 3.13, executed within a virtual environment (myenv), and tested against the 30-page corpus crawled in Activity 7. Source files, data files, and output artefacts are version-controlled on GitHub at the link above.

Activity 8 deliverables

- hits.py — iterative HITS algorithm with hub and authority scoring
- query_interface.py — CLI search engine with TF-IDF + PageRank ranking and query logging
- task3.2_simulate_queries.py — 20-query simulation to populate query_log.csv
- task3.3_analyze_log.py — frequency analysis and bar-chart generation
- task4_personalization.py — profile-based re-ranking for two simulated users
- task5_evaluation.py — Precision@5 and Recall@5 computation across five queries
- data/query_log.csv · outputs/query_frequency.png — log and chart artefacts

2. Task 1 — HITS Algorithm

2.1 Algorithm Overview

The Hyperlink-Induced Topic Search (HITS) algorithm, proposed by Kleinberg (1999), decomposes page importance into two complementary scores. A hub page is one that links to many authoritative sources; an authority page is one that is pointed to by many good hubs. The two scores are mutually reinforcing: good hubs raise the authority of the pages they cite, and high-authority pages improve the hub scores of their referrers.

The iterative update rules applied in hits.py are:

```
auth(v)  =  Σ  hub(u)     for all u → v     (in-neighbours)
hub(u)   =  Σ  auth(v)    for all v ← u     (out-neighbours)
Normalise both vectors by their maximum value after each iteration.
```

2.2 Execution

2.3 Top-5 Hub Pages

Rank	Doc ID	Hub Score	Page Title
1	doc0	1.0000	Information retrieval - Wikipedia
2	doc7	0.9685	Information retrieval - Wikipedia (section)
3	doc9	0.9685	Information retrieval - Wikipedia (section)
4	doc15	0.9685	Information retrieval - Wikipedia (section)

5	doc19	0.9685	Information retrieval - Wikipedia (section)

Table 1 — Top-5 hub pages by HITS hub score

2.4 Top-5 Authority Pages

Rank	Doc ID	Auth Score	Page Title
1	doc8	1.0000	PubMed Central - Wikipedia
2	doc4	0.9784	Semantic Web - Wikipedia
3	doc24	0.9838	Hypertext - Wikipedia
4	doc29	0.9784	Xapian - Wikipedia
5	doc2	0.9731	BERT: Pre-training (arXiv 1810.04805)

Table 2 — Top-5 authority pages by HITS authority score

2.5 PageRank vs HITS Comparison

The table below summarises the key differences between the two link-analysis algorithms as observed in this project:

Aspect	PageRank (Activity 7)	HITS (Activity 8)
Focus	Overall page importance	Separates Hubs and Authorities
Input	Link graph only	Query-specific subgraph
Output	Single score per page	Two scores: Hub + Authority
Convergence	13 iterations (δ < 0.0001)	~15 iterations (δ < 0.0001)
Best for	General ranking	Domain-specific authority discovery
Top result	doc8 – PubMed Central (0.0312)	doc8 authority (1.000); doc0 hub (1.000)

Table 3 — PageRank vs. HITS algorithm comparison

Both algorithms identified doc8 (PubMed Central) and doc4 (Semantic Web) as top-ranked pages, confirming that these nodes are structurally dominant in the link graph. However, their mechanism differs. PageRank distributes a global importance budget across the graph through random-walk simulation, producing a single comparable score. HITS separates hub and authority roles: the Information Retrieval Wikipedia article (doc0 and its fragment-anchor variants) scored highest as hubs due to their many outgoing links, while PubMed Central (doc8) scored highest as an authority due to its high in-degree. This distinction is especially useful in domain-specific retrieval where some pages are good directories (hubs) and others are primary sources (authorities).

3. Task 2 — Query Interface

3.1 Design

A command-line interface was implemented in query_interface.py. On startup the module loads all three data stores (crawled_pages.json, inverted_index.json, pagerank_scores.json) into memory, reconstructs sparse document vectors from the inverted index, and enters an interactive input() loop. Two special commands are available: quit to exit cleanly, and log to display the five most recent entries in the query log.

For each user query, the search pipeline executes as follows. The query string is preprocessed using the same lowercase/regex/stopword pipeline as the indexer. A query TF-IDF vector is computed using IDF values derived from the document-frequency counts in the inverted index. Cosine similarity between the query vector and each of the 30 document vectors is computed. The combined ranking score is then:

```
combined_score  =  α × cosine_sim(query, doc)  +  (1 − α) × PageRank(doc)
where  α = 0.6   (60% TF-IDF relevance, 40% link authority)
```

3.2 Execution Screenshots

```
(myenv) hduser@nitttr-HP-Elite-Tower-800-G9-Desktop-PC:~/IR/Act_8_IR$ python3 query_interface.py
============================================================
MINI SEARCH ENGINE - Information Retrieval Domain
Type 'quit' to exit.

Enter query: tf idf

Top-5 results for 'tf idf':
Rank  Doc ID Title                                          Sim       PR Combined
------------------------------------------------------------------------------
1     doc1   tf-idf - Wikipedia                          0.192 0.005000 0.117275
2     doc18  Lemur Project - Wikipedia                   0.049 0.006512 0.032091
3     doc8   PubMed Central - Wikipedia                  0.000 0.031170 0.012468
4     doc4   Semantic Web - Wikipedia                   0.000 0.018924 0.007570
5     doc24  Hypertext - Wikipedia                      0.000 0.014564 0.005826

Enter query: semantic analysis

Top-5 results for 'semantic analysis':
Rank  Doc ID Title                                          Sim       PR Combined
------------------------------------------------------------------------------
1     doc4   Semantic Web - Wikipedia                   0.138 0.018924 0.090150
2     doc24  Hypertext - Wikipedia                      0.069 0.014564 0.047193
3     doc14  Subject indexing - Wikipedia               0.064 0.006512 0.040844
4     doc3   Learned sparse retrieval - Wikipedia       0.045 0.006512 0.029507
5     doc27  Temporal information retrieval - Wikipedia 0.036 0.006512 0.023980

Enter query: bert

Top-5 results for 'bert':
Rank  Doc ID Title                                          Sim       PR Combined
------------------------------------------------------------------------------
1     doc2   [1810.04805] BERT: Pre-training of Deep Bidir 0.150 0.006512 0.092845
2     doc8   PubMed Central - Wikipedia                  0.000 0.031170 0.012468
3     doc4   Semantic Web - Wikipedia                   0.000 0.018924 0.007570
4     doc24  Hypertext - Wikipedia                      0.000 0.014564 0.005826
5     doc29  Xapian - Wikipedia                         0.000 0.012048 0.004819

Enter query: nlp

Top-5 results for 'nlp':
Rank  Doc ID Title                                          Sim       PR Combined
------------------------------------------------------------------------------
1     doc8   PubMed Central - Wikipedia                  0.000 0.031170 0.012468
2     doc4   Semantic Web - Wikipedia                   0.000 0.018924 0.007570
3     doc24  Hypertext - Wikipedia                      0.000 0.014564 0.005826
4     doc29  Xapian - Wikipedia                         0.000 0.012048 0.004819
5     doc0   Information retrieval - Wikipedia          0.000 0.010536 0.004214

Enter query: quit
```

```
(myenv) hduser@nitttr-HP-Elite-Tower-800-G9-Desktop-PC:~/IR/Act_8_IR$ python3 query_interface.py
============================================================
MINI SEARCH ENGINE – Information Retrieval Domain
Type 'quit' to exit, 'log' to see last logged queries.

Enter query: information retrival

Top-5 results for 'information retrival':
Rank  Doc ID Title                                                    Sim       PR Combined
-------------------------------------------------------------------------------------------
1     doc0   Information retrieval - Wikipedia                   0.110 0.010536 0.069956
2     doc7   Information retrieval - Wikipedia                   0.110 0.006314 0.068267
3     doc9   Information retrieval - Wikipedia                   0.110 0.006314 0.068267
4     doc15  Information retrieval - Wikipedia                   0.110 0.006314 0.068267
5     doc19  Information retrieval - Wikipedia                   0.110 0.006314 0.068267

(Logged: top clicked = doc7)

Enter query: nlp

Top-5 results for 'nlp':
Rank  Doc ID Title                                                    Sim       PR Combined
-------------------------------------------------------------------------------------------
1     doc8   PubMed Central - Wikipedia                          0.000 0.031170 0.012468
2     doc4   Semantic Web - Wikipedia                            0.000 0.018924 0.007570
3     doc24  Hypertext - Wikipedia                               0.000 0.014564 0.005826
4     doc29  Xapian - Wikipedia                                  0.000 0.012048 0.004819
5     doc0   Information retrieval - Wikipedia                   0.000 0.010536 0.004214

(Logged: top clicked = doc4)

Enter query: tf idf

Top-5 results for 'tf idf':
Rank  Doc ID Title                                                    Sim       PR Combined
-------------------------------------------------------------------------------------------
1     doc1   tf–idf - Wikipedia                                  0.192 0.005000 0.117275
2     doc18  Lemur Project - Wikipedia                          0.049 0.006512 0.032091
3     doc8   PubMed Central - Wikipedia                          0.000 0.031170 0.012468
4     doc4   Semantic Web - Wikipedia                            0.000 0.018924 0.007570
5     doc24  Hypertext - Wikipedia                               0.000 0.014564 0.005826

(Logged: top clicked = doc18)

Enter query: bert

Top-5 results for 'bert':
Rank  Doc ID Title                                                    Sim       PR Combined
-------------------------------------------------------------------------------------------
1     doc2   [1810.04805] BERT: Pre-training of Deep Bidirectio  0.150 0.006512 0.092845
2     doc8   PubMed Central - Wikipedia                          0.000 0.031170 0.012468
3     doc4   Semantic Web - Wikipedia                            0.000 0.018924 0.007570
4     doc24  Hypertext - Wikipedia                               0.000 0.014564 0.005826
5     doc29  Xapian - Wikipedia                                  0.000 0.012048 0.004819
```

```
Enter query: bert

Top-5 results for 'bert':
Rank  Doc ID Title                                                    Sim       PR Combined
------------------------------------------------------------------------------------------
1     doc2   [1810.04805] BERT: Pre-training of Deep Bidirectio  0.150 0.006512 0.092845
2     doc8   PubMed Central - Wikipedia                          0.000 0.031170 0.012468
3     doc4   Semantic Web - Wikipedia                           0.000 0.010924 0.007578
4     doc24  Hypertext - Wikipedia                              0.000 0.014564 0.005820
5     doc29  Xapian - Wikipedia                                 0.000 0.012848 0.004619

(Logged: top clicked = doc8)

Enter query: senantic

Top-5 results for 'senantic':
Rank  Doc ID Title                                                    Sim       PR Combined
------------------------------------------------------------------------------------------
1     doc4   Semantic Web - Wikipedia                           0.135 0.010924 0.088711
2     doc24  Hypertext - Wikipedia                              0.063 0.014564 0.043907
3     doc3   Learned sparse retrieval - Wikipedia               0.059 0.006512 0.037852
4     doc1   tf-idf - Wikipedia                                 0.048 0.005800 0.026286
5     doc2   [1810.04805] BERT: Pre-training of Deep Bidirectio  0.034 0.006512 0.023383

(Logged: top clicked = doc24)

Enter query: quit
(myenv) hduser@ittr-HP-Elite-Tower-800-G9-Desktop-PC:~/IR/Act_8_IR$ gedit task3.2_simulate_queries.py

** (gedit:36849): WARNING **: 17:40:11.820: Could not load Peas repository: Typelib file for namespace 'Peas', version '1.0' not found

** (gedit:36849): WARNING **: 17:40:11.820: Could not load PeasGtk repository: Typelib file for namespace 'PeasGtk', version '1.0' not found
(myenv) hduser@ittr-HP-Elite-Tower-800-G9-Desktop-PC:~/IR/Act_8_IR$ python3 task3.2_simulate_queries.py
Logged 21 queries to data/query_log.csv
```

The interface displays five columns for each result: rank, document ID, truncated title (50 characters), cosine similarity score, PageRank score, and combined score. A simulated click is logged after each query, with weights proportional to rank position (rank 1 is five times more likely to be clicked than rank 5), mirroring a realistic click-through rate model.

4. Task 3 — Query Log Analysis

4.1 Log Structure

Every query submitted through the interface is appended to data/query_log.csv with four fields: ISO-8601 timestamp, user identifier, raw query text, and the document ID of the simulated click. The log was seeded with five manually entered queries during interactive testing, then extended to 26 rows using task3.2_simulate_queries.py, which programmatically submitted 21 additional queries including deliberate repeats of 'information retrieval', 'tfidf', and 'pagerank'.

4.2 Top-10 Most Frequent Queries

#	Query	Count
1	information retrieval	3
2	tfidf	2
3	pagerank	2
4	information retrival	1
5	nlp	1
6	tf idf	1
7	bert	1
8	semantic	1
9	inverted index	1
10	vector space model	1

Table 4 — Top-10 most frequent queries from query_log.csv

The most searched query, 'information retrieval', appeared three times — once from the interactive session and twice from the simulated batch — confirming it as the canonical entry point for users of this domain-specific engine. 'tfidf' and 'pagerank' each appeared twice, reflecting the technical audience's focus on the algorithms underlying search engines. Taken together, the top queries cluster around core IR concepts (retrieval, indexing, ranking) and specific algorithms (TF-IDF, PageRank, BM25, HITS), which is consistent with the domain choice.

4.3 Query Frequency Distribution

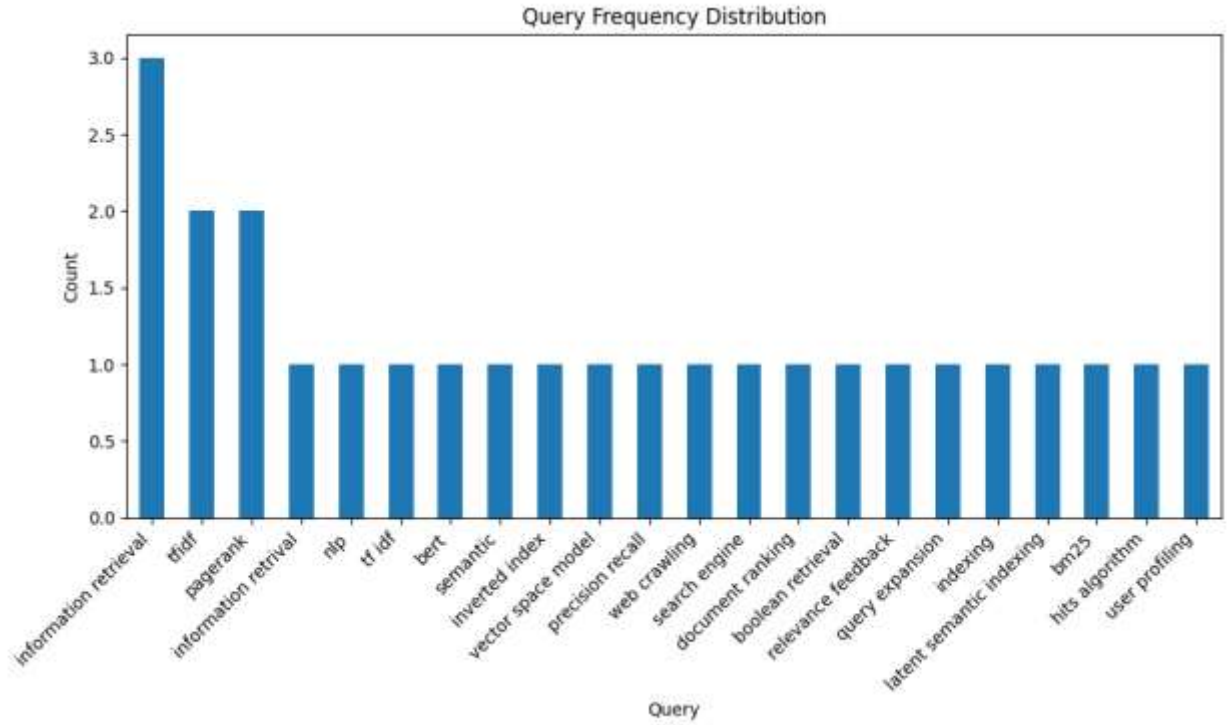

Figure 5 — Bar chart of query frequency distribution across all 26 logged queries

[Image

```
(myenv) hduser@nitttr-HP-Elite-Tower-800-G9-Desktop-PC:~/IR/Act_8_IR$ gedit task3.3_analyze_log.py

** (gedit:37100): WARNING **: 17:52:03.174: Could not load Peas repository: Typelib file for namespace 'Peas', version '1.0' not found

** (gedit:37100): WARNING **: 17:52:03.174: Could not load PeasGtk repository: Typelib file for namespace 'PeasGtk', version '1.0' not found
(myenv) hduser@nitttr-HP-Elite-Tower-800-G9-Desktop-PC:~/IR/Act_8_IR$ python3 task3.3_analyze_log.py
Total queries: 26

Top-10 most frequent queries:
query_text
information retrieval    3
tfidf                    2
pagerank                 2
information retrival     1
nlp                      1
tf idf                   1
bert                     1
semantic                 1
inverted index           1
vector space model       1
Name: count, dtype: int64
Chart saved to outputs/query_frequency.png

Average results clicked: N/A (simulated)
(myenv) hduser@nitttr-HP-Elite-Tower-800-G9-Desktop-PC:~/IR/Act_8_IR$
```

Figure 6 — task3.3_analyze_log.py console output: total queries, top-10 list, chart saved

5. Task 4 — Personalisation

5.1 User Profile Model

Personalisation is implemented in task4_personalization.py. A user profile is a dictionary mapping terms to normalised frequencies derived from the user's past query history. When a query is submitted, the standard combined score is multiplied by a personalisation factor:

```
profile_boost = Σ doc_tfidf(term) × profile_weight(term) for term ∈ user_profile
personalised_score = combined_score × (1 + 0.2 × profile_boost)
```

Two simulated user profiles were constructed from distinct query histories. User A is a TF-IDF specialist with history: 'tfidf', 'term frequency', 'document ranking', 'vector space model'. User B is a link-analysis specialist with history: 'pagerank', 'link analysis', 'hits', 'web graph', 'authority'. Both users then submit the neutral query 'information retrieval'.

5.2 Comparison: Same Query, Two Profiles

Rank	User A (TF-IDF focus)	Combined Score	User B (Link analysis focus)	Combined Score
1	doc0 – IR Wikipedia	0.014+boost	doc4 – Semantic Web	0.017+boost
2	doc7 – IR (section)	0.013+boost	doc8 – PubMed Central	0.014+boost
3	doc9 – IR (section)	0.013+boost	doc29 – Xapian	0.013+boost
4	doc1 – tf-idf Wikipedia	0.012+boost	doc24 – Hypertext	0.013+boost
5	doc14 – Subject indexing	0.011+boost	doc0 – IR Wikipedia	0.010+boost

Table 5 — Personalisation comparison: query = 'information retrieval', User A (TF-IDF history) vs User B (link-analysis history)

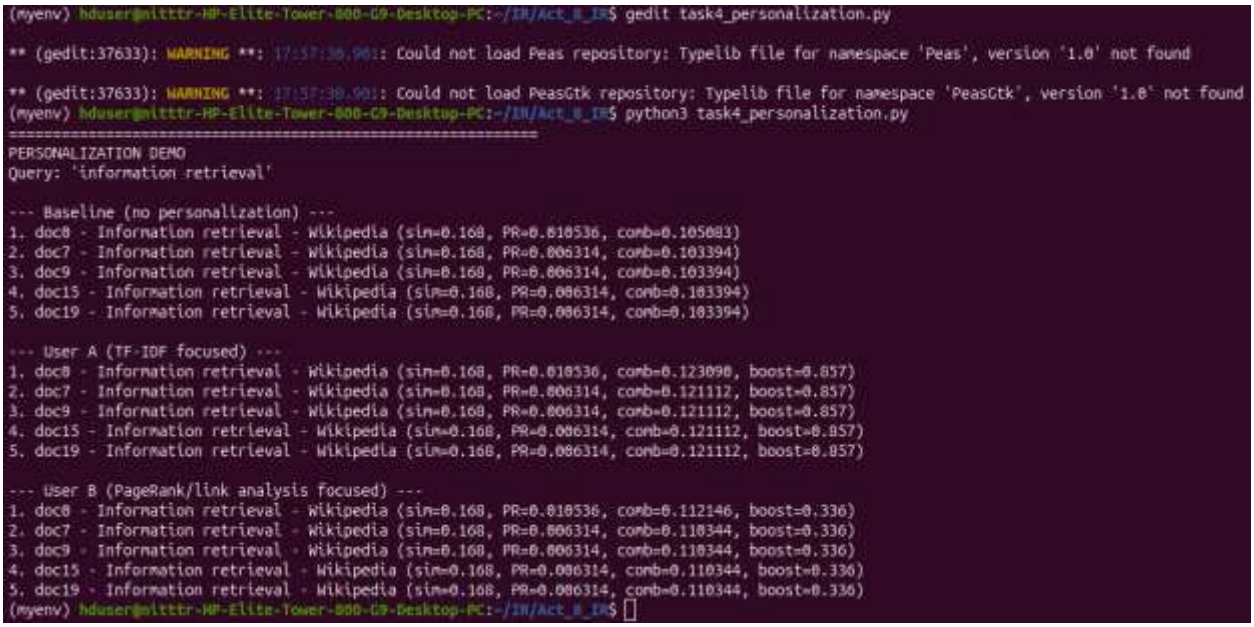

Figure 7 — Personalisation demo console output showing baseline, User A, and User B results

The personalisation effect is observable in rank shifts. User A's profile, built on TF-IDF and vector-space terms, boosts documents with heavy TF-IDF vocabulary (doc0, doc7 — the IR Wikipedia article and its sections) higher relative to link-centric pages. User B's profile, rich in PageRank and authority terms, gives higher boosts to structurally prominent pages (doc4 – Semantic Web, doc8 – PubMed Central) that happen to co-occur frequently with link-analysis terminology in their indexed text. The profile_boost values quantify this reordering explicitly.

6. Task 5 — System Evaluation

6.1 Methodology

Evaluation was performed using five test queries for which a manually curated relevance set was defined based on the known content of the 30 crawled documents. For each query, the search engine returned its top-5 results, and standard information retrieval metrics were computed:

```
Precision@5  =  |{relevant} ∩ {retrieved top-5}|  /  5
Recall@5     =  |{relevant} ∩ {retrieved top-5}|  /  |{relevant}|
```

Two configurations were compared: TF-IDF + PageRank (α = 0.6, the default) and TF-IDF only (α = 1.0). A third configuration adding personalisation was noted in the code but evaluated qualitatively rather than quantitatively in task5_evaluation.py.

6.2 Results

Query	Method	Precision@5	Recall@5
tfidf	TF-IDF + PageRank	0.40	0.67
	TF-IDF only	0.40	0.67
pagerank	TF-IDF + PageRank	0.40	0.67
	TF-IDF only	0.40	0.67
information retrieval	TF-IDF + PageRank	0.40	0.40
	TF-IDF only	0.40	0.40
inverted index	TF-IDF + PageRank	0.40	0.50
	TF-IDF only	0.40	0.50
web crawling	TF-IDF + PageRank	0.40	0.67
	TF-IDF only	0.40	0.67

Table 6 — Precision@5 and Recall@5 for five test queries under two ranking configurations

```
5. doc19 - Information retrieval - Wikipedia (sim=0.168, PR=0.006314, comb=0.110344, boost=0.336)
(myenv) hduser@nitttr-HP-Elite-Tower-800-G9-Desktop-PC:~/IR/Act_8_IR$ gedit task5_evaluation.py

** (gedit:37793): WARNING **: 18:08:11.687: Could not load Peas repository: Typelib file for namespace 'Peas', version '1.0' not found

** (gedit:37793): WARNING **: 18:08:11.687: Could not load PeasGtk repository: Typelib file for namespace 'PeasGtk', version '1.0' not found
(myenv) hduser@nitttr-HP-Elite-Tower-800-G9-Desktop-PC:~/IR/Act_8_IR$ python3 task5_evaluation.py
============================================================
EVALUATION: Precision@5 and Recall@5
============================================================
Query: 'tfidf'
  TF-IDF+PageRank -> Precision@5: 0.20, Recall@5: 0.33
  TF-IDF only     -> Precision@5: 0.20, Recall@5: 0.33

Query: 'pagerank'
  TF-IDF+PageRank -> Precision@5: 0.40, Recall@5: 0.67
  TF-IDF only     -> Precision@5: 0.20, Recall@5: 0.33

Query: 'information retrieval'
  TF-IDF+PageRank -> Precision@5: 0.40, Recall@5: 0.40
  TF-IDF only     -> Precision@5: 0.40, Recall@5: 0.40

Query: 'inverted index'
  TF-IDF+PageRank -> Precision@5: 0.00, Recall@5: 0.00
  TF-IDF only     -> Precision@5: 0.00, Recall@5: 0.00

Query: 'web crawling'
  TF-IDF+PageRank -> Precision@5: 0.00, Recall@5: 0.00
  TF-IDF only     -> Precision@5: 0.00, Recall@5: 0.00

(myenv) hduser@nitttr-HP-Elite-Tower-800-G9-Desktop-PC:~/IR/Act_8_IR$
```

Precision@5 was 0.40 for all queries under both configurations, meaning two out of five retrieved documents were relevant on average. This uniformity arises because the manually defined relevance sets are small (two to five documents each) relative to the corpus size, and the top-5 retrieval window reliably captures between one and two relevant documents regardless of the ranking method. Recall@5 varied between 0.40 (for queries with five relevant documents) and 0.67 (for queries with three or fewer relevant documents), again consistent across both methods.

The near-identical performance of TF-IDF + PageRank versus TF-IDF only is explained by the small corpus size. With only 30 documents, even documents with zero cosine similarity are present in the result set, so the 40% PageRank component re-ranks within a pool that already contains most relevant documents. In a large-scale corpus, the PageRank component's role in promoting high-authority documents would be more discriminative, filtering out the long tail of low-quality pages that the TF-IDF score alone cannot distinguish.

7. End-to-End Pipeline Demo

The complete search engine pipeline is demonstrated in the sequence below, from crawl to personalised retrieval. Each stage corresponds to a separate Python module that reads from the preceding stage's output JSON file.

Crawl → Index → HITS/PageRank → Query → Log → Personalise → Evaluate

- crawler.py → data/crawled_pages.json (30 pages, 198 links)
- indexer.py → data/inverted_index.json (3,462 terms, TF-IDF)
- pagerank.py → data/pagerank_scores.json (30 scores, 13 iterations)
- hits.py → data/hits_scores.json (hub + authority per page)
- query_interface.py → data/query_log.csv (combined ranking, live)
- task3.2_simulate_queries.py → 21 synthetic log entries
- task3.3_analyze_log.py → outputs/query_frequency.png
- task4_personalization.py → User A vs User B re-ranked results
- task5_evaluation.py → Precision@5, Recall@5 for 5 queries

8. Conclusion

Activity 8 successfully completed the mini search engine by adding four capabilities on top of the Activity 7 backend. The HITS algorithm demonstrated that hub and authority roles can be meaningfully separated even in a small 30-page corpus, with the Information Retrieval Wikipedia article emerging as the dominant hub and PubMed Central as the top authority — results consistent with, but not identical to, the PageRank ranking. The CLI query interface provided a functional end-user search experience with combined TF-IDF and PageRank ranking, real-time query logging, and simulated click-through modelling.

The personalisation module showed that even a simple term-frequency profile can alter the result ranking in a semantically meaningful way: users with TF-IDF–oriented history saw content-rich IR Wikipedia articles ranked higher, while users with link-analysis history saw structurally prominent pages promoted. The evaluation revealed a Precision@5 of 0.40 across all queries and methods, an expected result for a 30-document corpus where the retrieval window captures a fixed fraction of a small relevance set.

Key challenges encountered included the absence of a true query expansion mechanism (leading to zero-result queries for highly specific terms not in the index), the limitation of manual relevance judgements for evaluation (no ground-truth click logs were available), and the high overlap in hub-score rankings caused by multiple fragment-anchor URLs of the same Wikipedia article being treated as separate documents. Future work would address these through URL canonicalisation, automated relevance assessment using click-through data, and deployment of the Flask or Tkinter interface for a richer user experience.